Project - Low Level Design

On

CI/CD Pipeline For Python Energy Management Application Using Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Arnika Jain	EN22CS301196
2.	Aryan Singh	EN22CS301214
3.	Ansh Upadhyay	EN22CS301147
4.	Arnoday Soni	EN22CS301197
5.	Anuroop Patidar	EN22CS301176

Group Name: Group 01-D2

Project Number: DO-13

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Table of Contents

1. Introduction.
    1.1. Scope of the document
    1.2. Intended Audience
    1.3. System overview.

2. System Design.
    2.1. Application Design
    2.2. Process Flow.
    2.3. Information Flow.
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue.

3. References

1. Introduction

The Energy management application is an AI-powered research platform designed to assist users in analyzing energy-related topics. The system uses Agentic AI and Retrieval-Augmented Generation (RAG) techniques to collect, analyze, and generate structured research reports on energy topics.

The system integrates DevOps practices to automate the development lifecycle. Tools such as Git, Docker, Jenkins/GitHub Actions, and Kubernetes are used to implement Continuous Integration and Continuous Deployment (CI/CD). This enables automated building, testing, containerization, and deployment of the application.

The platform allows users to submit research queries related to energy topics, upload documents, and receive structured AI-generated reports.

1.1 Scope of the Document

This document provides a detailed Low Level Design of the Autonomous Energy Researcher Agent system with DevOps integration**.**

The document includes:

- System architecture
- Application modules
- Process and information flow
- Component-level design
- API catalogue
- DevOps pipeline design

It also describes how the application is containerized using Docker and deployed using Kubernetes through automated CI/CD pipelines.

1.2 Intended Audience

This document is intended for:

- Software developers
- DevOps engineers
- System architects
- Project reviewers
- Students and researchers working on AI-based systems

The document helps stakeholders understand the internal design, components, and deployment workflow of the system.

1.3 System Overview

The Autonomous Energy Researcher Agent is a multi-agent AI system designed to perform automated research on energy-related topics.

The system architecture includes the following components:

1. Frontend Interface
    - Developed using Streamlit
    - Allows users to submit queries and view reports
2. Backend API
    - Built using FastAPI
    - Handles application logic and communication between modules
3. Agentic AI System
    - Built using LangChain or CrewAI
    - Includes multiple agents responsible for research tasks
4. Knowledge Base
    - Uses FAISS vector database
    - Stores embeddings of uploaded documents
5. LLM Integration
    - Uses Groq LLM (Mixtral model) for report generation
6. DevOps Infrastructure
    - Git for version control
    - Docker for containerization
    - Jenkins or GitHub Actions for CI/CD
    - Kubernetes for deployment

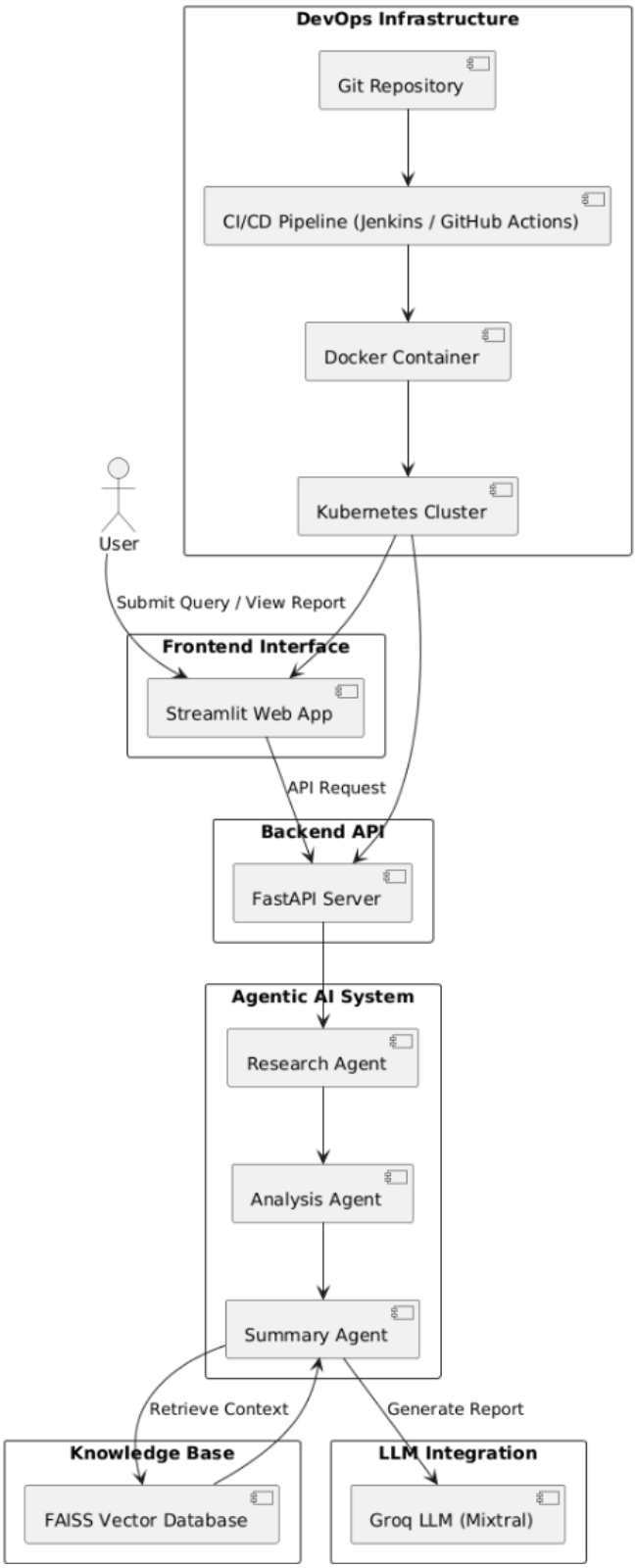

Figure 1: Use Case Diagram

2. System Design

The system design focuses on modular architecture and automated deployment using DevOps pipelines.

2.1 Application Design

The application follows a three-layer architecture.

1. Presentation Layer

This layer provides the user interface where users interact with the system.

Technologies used:

- Streamlit
- HTML/CSS (optional components)

Responsibilities:

- Accept research queries
- Display AI-generated reports
- Upload documents for RAG

---

2. Application Layer

This layer contains the business logic of the system.

Technologies used:

- Python
- FastAPI

Responsibilities:

- Query processing
- Agent orchestration
- Report generation
- API communication

---

3. Data Layer

This layer stores processed information and document embeddings.

Technologies used:

- FAISS vector database
- Local storage or cloud storage

Responsibilities:

- Store embeddings
- Retrieve relevant documents for RAG

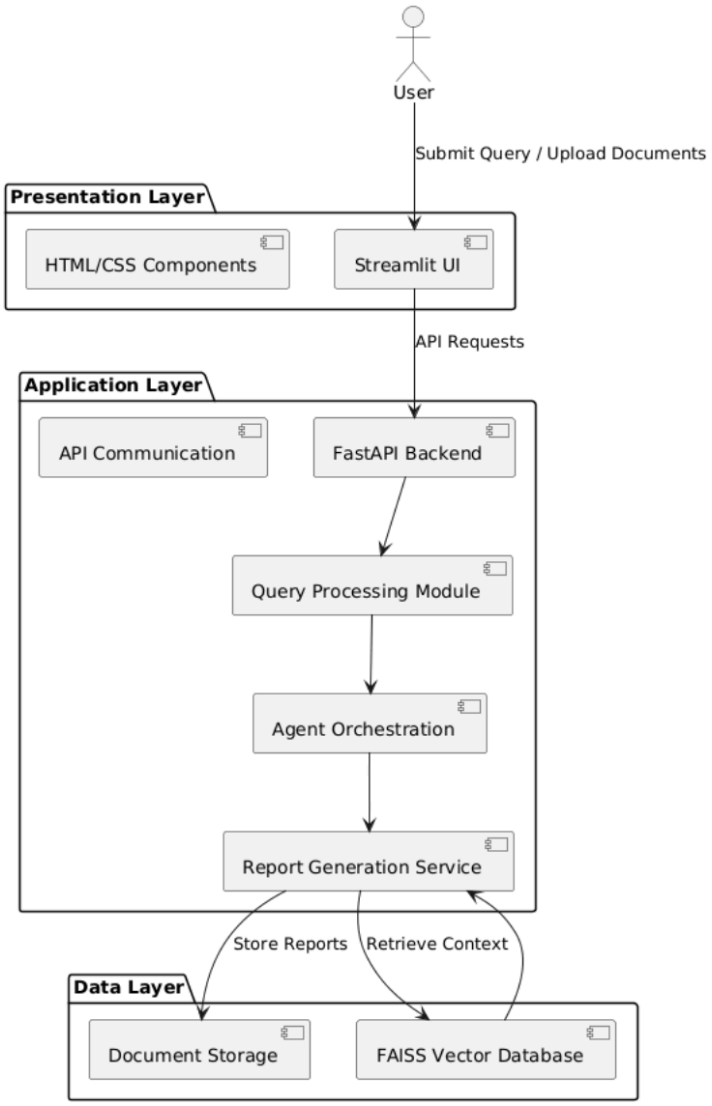

Figure 2.1: Architecture Diagram

2.2 Process Flow

The process flow describes how a user query is processed by the system.

Step 1

User enters a research query in the Streamlit interface.

Step 2

The query is sent to the FastAPI backend.

Step 3

The backend triggers the Agentic AI system**.**

Step 4

Multiple AI agents perform tasks such as:

- Web search
- Data extraction
- Content analysis
- Report generation

Step 5

The system retrieves relevant documents from the FAISS vector database.

Step 6

The LLM generates a structured research report.

Step 7

The generated report is displayed on the user interface.

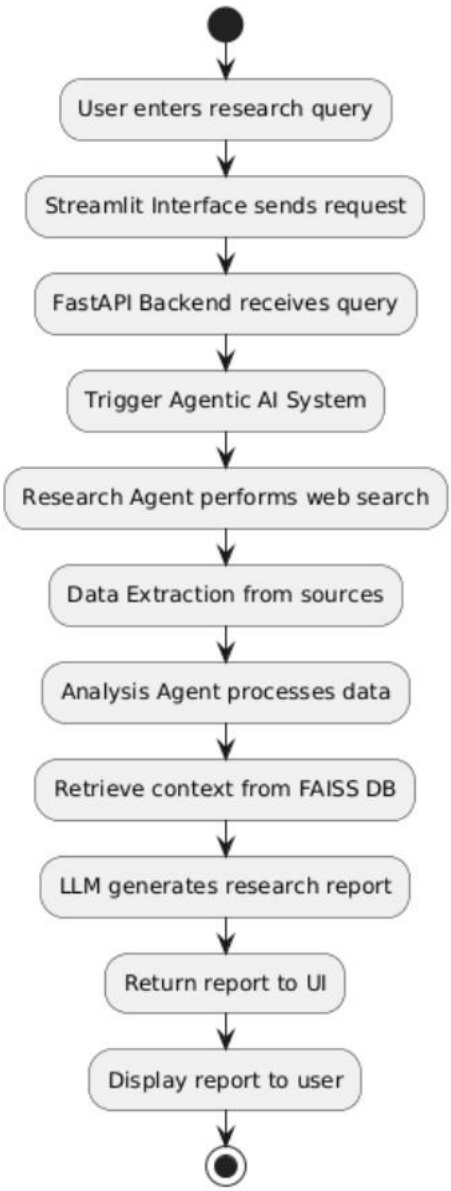

Figure 2.2: Activity Diagram

2.3 Information Flow

The information flow within the system is as follows:

User → Streamlit Interface → FastAPI Backend → Agentic AI System → Vector Database → LLM → Report Generation → User Interface

This ensures that user queries are processed efficiently and the results are returned in a structured format.

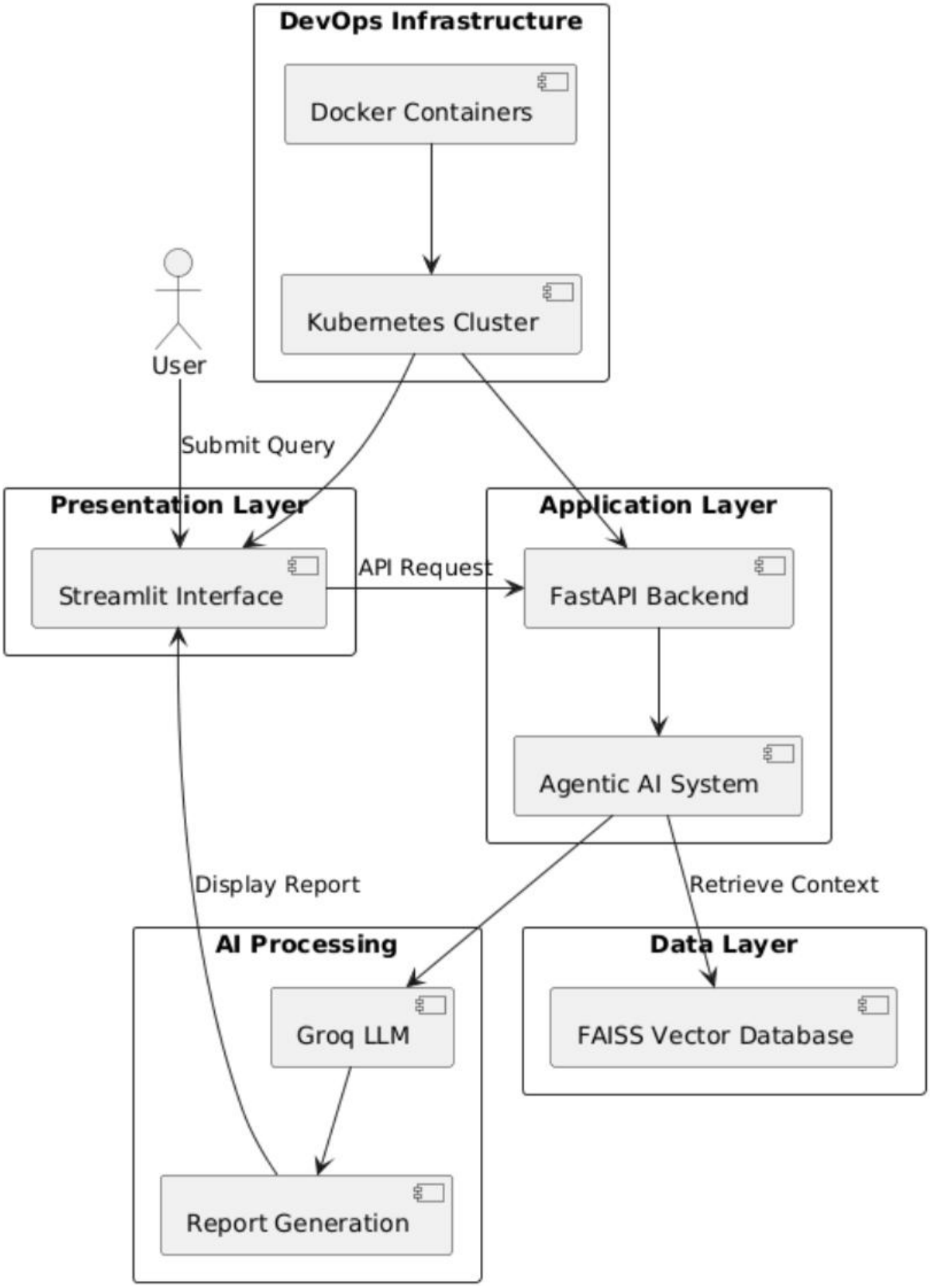

Figure 2.3: Data Flow Diagram

2.4 Components Design

The system consists of the following components.

1. User Interface Module

Responsibilities:

- Accept research queries
- Display reports
- Upload documents

Technology:

- Streamlit

---

2. Backend API Module

Responsibilities:

- Process requests
- Manage agent communication
- Handle API endpoints

Technology:

- FastAPI

---

3. Agentic AI Module

This module contains multiple autonomous agents responsible for performing research tasks.

Types of agents:

1. **Search Agent**
   - Searches energy-related information online.
2. **Scraping Agent**
   - Extracts data from websites.
3. **Analysis Agent**
   - Processes and summarizes data.
4. **Report Generation Agent**
   - Generates structured research reports.

Technology:

- LangChain / CrewAI

---

4. RAG Module

This module handles document-based knowledge retrieval.

Responsibilities:

- Document upload
- Text chunking
- Embedding generation
- Vector storage
- Context retrieval

Technology:

- FAISS

---

5. DevOps Module

This module automates the deployment pipeline.

Components:

1. **Git Repository**
   - Stores source code.
2. **CI/CD Pipeline**
   - Automates testing and building.
3. **Docker**
   - Containerizes the application.
4. **Kubernetes**
   - Manages container orchestration.

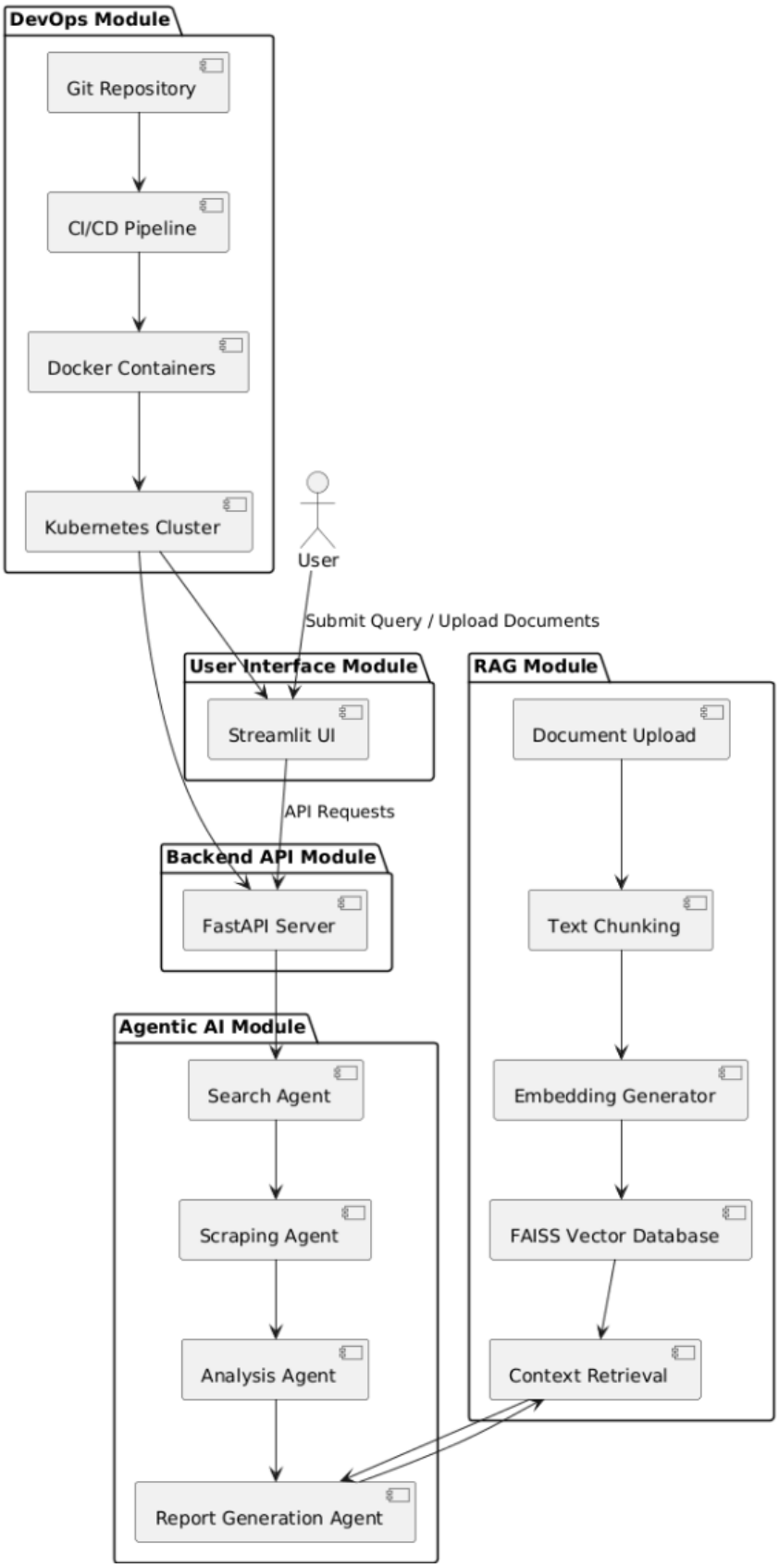

Figure 2.4: Component Diagram

2.5 Key Design Considerations

The following design principles are considered in this system.

Scalability

Kubernetes allows horizontal scaling of application containers.

Reliability

Container orchestration ensures system reliability.

Modularity

The system is divided into independent modules for better maintainability.

Automation

CI/CD pipelines automate build and deployment processes.

Performance

Vector database improves the speed of document retrieval.

2.6 API Catalogue

The system provides several API endpoints for interaction between modules.

1. Submit Research Query

Endpoint
POST /generate-report

Description
Generates an AI research report based on the user query.

---

2. Upload Document

Endpoint
POST /upload-document

Description
Uploads energy-related documents for RAG processing.

3. Chat with Documents

Endpoint
POST /chat

Description
Allows users to ask questions based on uploaded documents.

4. Get Reports

Endpoint
GET /reports

Description
Retrieves previously generated research reports.

3. References

1. LangChain Documentation
2. CrewAI Documentation
3. FastAPI Documentation
4. Docker Documentation
5. Kubernetes Documentation
6. Jenkins Documentation
7. GitHub Actions Documentation
8. FAISS Vector Database Documentation
9. Groq LLM API Documentation